

Relatório Técnico — Reconhecimento Facial com Eigenfaces (PCA/SVD)

Projeto final — Modelos Aproximados / Computação Científica Autor: Daniel — Ciência da Computação, UFRJ (Instituto de Computação)

1. Introdução e objetivo

Este projeto implementa um sistema de reconhecimento facial simplificado usando a técnica clássica de **Eigenfaces** (Turk & Pentland, 1991), aplicando os conceitos de **PCA (Análise de Componentes Principais)** e **decomposição SVD** estudados na disciplina — autovalores e autovetores, ortogonalidade, aproximação de posto baixo, mínimos quadrados e estabilidade numérica.

Dataset: Olivetti Faces — 40 pessoas, 10 fotos cada (400 imagens no total), 64×64 pixels em escala de cinza.

Objetivo funcional: dado um conjunto de fotos de treino de 40 pessoas conhecidas, identificar corretamente a identidade de fotos novas (teste) dessas mesmas pessoas, comprimindo cada rosto (4096 pixels) num vetor pequeno de coeficientes antes de comparar por distância.

2. Estrutura do repositório

```
eigenfaces/
├── README.md
├── requisitos.txt
├── .gitignore
├── dados/                # dataset em cache (ignorado no git)
├── src/
│   ├── carregar_dados.py # carrega o dataset Olivetti em matriz A
│   └── pca.py             # PCA via autovalores de  $A^T A$ , projeção,
reconstrução
│   └── eigenfaces.py      # gera eigenfaces, reconstrução, variância
explicada
│   ├── reconhecer.py      # classificação por vizinho mais próximo
│   ├── matriz_confusao.py # matriz de confusão da classificação
│   └── comparar_svd.py    # teste de sanidade: PCA via  $A^T A$  vs. SVD
direta
├── notebooks/
│   └── eigenfaces_completo.ipynb # notebook consolidado, narrativo, 27
células
├── resultados/           # gráficos/imagens gerados
├── relatorio/             # este relatório e material de
apresentação
```

3. Fundamentação matemática

3.1 Representação dos dados

Cada imagem (64×64 pixels) é "esticada" num vetor coluna de 4096 números. Empilhando as 400 fotos, monta-se a matriz:

$$A \in \mathbb{R}^{4096 \times 400}$$

onde cada **coluna** representa um rosto. Essa é a mesma estrutura matricial usada nos exercícios de PRM da disciplina, só que em escala muito maior.

3.2 Centralização (subtração da média)

PCA busca as direções de **máxima variância** dos dados, e variância é definida em relação à média. Por isso, antes de qualquer cálculo, obtém-se o "rosto médio" — a média de cada pixel ao longo das 400 fotos — e subtrai-se esse vetor de cada coluna:

$$\bar{a} = \frac{1}{n} \sum_{i=1}^n a_i \quad A_c = A - \bar{a} \mathbf{1}^T$$

Sem essa centralização, a "direção de maior variação" ficaria contaminada pela posição do centro de massa dos dados, e não representaria a variação real entre rostos.

3.3 PCA via autovalores de $A^T A$ — o núcleo do projeto

Formalmente, PCA busca os autovetores da matriz de covariância:

$$C = \frac{1}{n-1} A_c A_c^T \in \mathbb{R}^{4096 \times 4096}$$

Calcular autovalores/autovetores diretamente dessa matriz (4096×4096, ~16 milhões de entradas) é computacionalmente caro. Como $n = 400 \ll 4096$, é muito mais eficiente trabalhar com a matriz pequena:

$$M = \frac{1}{n-1} A_c^T A_c \in \mathbb{R}^{400 \times 400}$$

Prova da equivalência: se v é autovetor de $A_c^T A_c$ com autovalor λ :

$$A_c^T A_c v = \lambda v$$

Multiplicando os dois lados por A_c à esquerda:

$$A_c A_c^T (A_c v) = \lambda (A_c v)$$

Logo, $(A_c v)$ é autovetor de $A_c A_c^T$ associado ao **mesmo autovalor** λ . Resolve-se o problema pequeno (400×400), e recuperam-se os autovetores grandes (as eigenfaces de 4096 números) multiplicando por A_c .

Detalhe crítico de implementação: o vetor resultante $A_c v$ **não tem norma 1** automaticamente — é necessário normalizar explicitamente após essa multiplicação.

Por que `np.linalg.eigh` e não `np.linalg.eig`: $M = A_c^T A_c$ é sempre simétrica (e semidefinida positiva). `eigh` é especializado para matrizes simétricas: mais rápido, mais estável numericamente, e garante autovalores reais.

3.4 Por que a normalização é $1/(n-1)$ e não $1/n$ — Correção de Bessel

Sem normalização, $A_c A_c^T$ (chamada de matriz de dispersão) cresce artificialmente com o número de amostras n , tornando os autovalores não-comparáveis entre datasets de tamanhos diferentes.

A divisão por $n - 1$ (em vez de n) é a **correção de Bessel**: como a média usada na centralização foi calculada a partir dos próprios dados (não é a média populacional verdadeira), ela "gasta" um grau de liberdade — restam $n - 1$ graus de liberdade independentes para estimar a variância. Dividir por n produziria um estimador viesado (subestimaria sistematicamente a variância real).

Ponto sutil importante: multiplicar uma matriz por uma constante positiva não muda seus autovetores — só reescala os autovalores pela mesma constante ($Mv = \lambda v \Rightarrow (cM)v = c\lambda v$). Ou seja, **as eigenfaces seriam idênticas** independente da normalização escolhida; o que muda é só o valor numérico dos autovalores, relevante para a interpretação de variância explicada e para a fórmula de conversão com valores singulares da SVD (seção 3.7).

3.5 Eigenfaces

Cada coluna da matriz de autovetores resultante é uma "eigenface" — uma direção no espaço de 4096 dimensões. Ao remontar como imagem 64×64 , aparecem padrões reconhecíveis (contorno de óculos, iluminação, formato geral do rosto) — mas **eigenfaces não são rostos reais nem características interpretáveis no sentido humano**; são direções matemáticas abstratas, otimizadas para capturar o máximo de variância estatística possível.

3.6 Variância explicada

Cada autovalor λ_i representa quanta variância está capturada naquela direção. A variância explicada acumulada com os primeiros k componentes é:

$$\frac{\sum_{i=1}^k \lambda_i}{\sum_{i=1}^n \lambda_i}$$

Resultado obtido: ~38 componentes capturam 95% da variância total; ~177 capturam 99% — de um total de até **399** componentes possíveis (não 400: a centralização introduz uma restrição linear entre as colunas — a soma das colunas centralizadas é sempre zero — reduzindo o posto máximo em exatamente 1).

3.7 Reconstrução e o Teorema de Eckart-Young

Projeção (rosto $\rightarrow$ coeficientes):

$$c = V_k^T (a - \bar{a})$$

Reconstrução (coeficientes $\rightarrow$ rosto):

$$\hat{a} = V_k c + \bar{a}$$

O **Teorema de Eckart-Young** garante que a truncagem do PCA/SVD nos k maiores componentes é a **melhor aproximação de posto k possível** de uma matriz, na norma de Frobenius. Consequência direta: o erro de reconstrução deve ser **monotonicamente não-crescente** conforme k aumenta — usar mais informação nunca pode piorar a melhor aproximação possível.

3.8 Relação com SVD

Toda matriz admite $A = U\Sigma V^T$. As colunas de U são as eigenfaces (autovetores de AA^T), e a relação entre valor singular σ_i e autovalor da covariância é:

$$\lambda_i = \frac{\sigma_i^2}{n-1}$$

Autovetores (e colunas de U) são definidos **a menos de sinal** — v e $-v$ são igualmente válidos — por isso qualquer comparação entre as duas rotas exige alinhar os sinais antes.

3.9 Número de condição e estabilidade numérica

Fato teórico central: $\text{cond}(A^T A) = \text{cond}(A)^2$. Formar $A^T A$ explicitamente (como a implementação deste projeto faz) **eleva ao quadrado** a sensibilidade a erro numérico, comparado a trabalhar diretamente com A (como a SVD direta faz, sem nunca formar $A^T A$).

Resultado empírico obtido:

- $\text{cond}(A) \approx 4,4 \times 10^5$
- $\text{cond}(A^T A) \approx 1,93 \times 10^{11}$ (bate com $\text{cond}(A)^2$)
- Diferença numérica observada entre as duas rotas: $\sim 10^{-6}$
- Estimativa teórica: precisão de máquina ($\approx 10^{-16}$) $\times$ número de condição ($\approx 10^{11}$) $\approx 10^{-5}$ — mesma ordem de grandeza do medido.

Conclusão: a implementação via $A^T A$ é mais rápida (confirmado empiricamente, cerca de 3-6× mais rápida nos testes) mas teoricamente menos estável numericamente que SVD direta. Para este dataset, a imprecisão é desprezível — uma troca de engenharia razoável e conscientemente justificada.

4. Pipeline de implementação

4.1 Carregamento dos dados (`carregar_dados.py`)

```
def carregar_matriz_rostos():
    dados = fetch_olivetti_faces()
    imagens = dados.images
    rotulos = dados.target
    altura, largura = imagens.shape[1], imagens.shape[2]
    A = imagens.reshape(imagens.shape[0], -1).T
    return A, rotulos, altura, largura
```

4.2 Núcleo do PCA (`pca.py`)

```

def calcular_pca(A, n_componentes=None):
    n_amostras = A.shape[1]
    if n_componentes is None:
        n_componentes = n_amostras
    M = (A.T @ A) / (n_amostras - 1)
    autovalores, autovetores_M = np.linalg.eigh(M)
    ordem = np.argsort(autovalores)[::-1]
    autovalores = autovalores[ordem]
    autovetores_M = autovetores_M[:, ordem]
    autovetores = A @ autovetores_M
    normas = np.linalg.norm(autovetores, axis=0)
    normas[normas == 0] = 1
    autovetores = autovetores / normas
    n_componentes = min(n_componentes, autovetores.shape[1])
    return autovetores[:, :n_componentes], autovalores[:n_componentes]

def projetar(A, media, autovetores):
    A_centralizada = A - media[:, None]
    return autovetores.T @ A_centralizada

def reconstruir(coeficientes, media, autovetores):
    return autovetores @ coeficientes + media[:, None]

def variancia_explicada_acumulada(autovalores):
    autovalores = np.clip(autovalores, a_min=0, a_max=None)
    total = autovalores.sum()
    return np.cumsum(autovalores) / total if total > 0 else
    np.zeros_like(autovalores)

```

Nota importante de design: `calcular_pca` com `n_componentes=None` retorna **todos** os componentes possíveis de uma vez (até 399). A redução de dimensionalidade em si é um passo **separado e posterior**, feito simplesmente fatiando as primeiras k colunas do resultado (`autovetores[:, :k]`). Ou seja, $A^T A$ resolve o problema de **eficiência computacional**; a escolha de quantos componentes manter resolve o problema de **compressão/redução de dimensionalidade** — são dois problemas distintos, frequentemente confundidos por acontecerem na mesma função.

4.3 Classificação por vizinho mais próximo (`reconhecer.py`)

```

def classificar_vizinho_mais_proximo(coef_treino, rotulos_treino,
coef_teste):
    previsoes = []
    for i in range(coef_teste.shape[1]):
        vetor_teste = coef_teste[:, i]
        distancias = np.linalg.norm(coef_treino - vetor_teste[:, None],
axis=0)
        previsoes.append(rotulos_treino[np.argmin(distancias)])
    return np.array(previsoes)

```

Metodologia (evitando vazamento de dados / data leakage):

- Divisão 75% treino / 25% teste, **estratificada** (`stratify=rotulos`) — garante que cada uma das 40 pessoas tem representação proporcional em ambos os conjuntos.
- As eigenfaces são calculadas **exclusivamente com o treino** (`calcular_pca(A_treino_centralizada, ...)`).
- O teste é projetado usando a **média e as eigenfaces do treino** (`projetar(A_teste, media_treino, autovetores_treino)`) — nunca recalcula PCA a partir do teste.

O que exatamente o treino "ensina": as eigenfaces e a média calculadas no treino são parâmetros aprendidos que representam a melhor estimativa das direções de variação real entre rostos, assumindo que treino e teste vêm da mesma população estatística (as mesmas 40 pessoas, fotos diferentes). O teste avalia se essa base generaliza bem o suficiente para representar fotos novas das mesmas pessoas, de forma que a distância euclidiana no espaço reduzido continue refletindo semelhança facial real — **não** avalia reconhecimento de pessoas nunca vistas (ver seção 6, Limitações).

Resultado obtido: 94% de acurácia com 50 componentes (100 fotos de teste, 6 erros). O experimento de acurácia vs. número de componentes mostrou saturação rápida a partir de ~20-30 componentes.

4.4 Matriz de confusão (`matriz_confusao.py`)

```
matriz = confusion_matrix(rotulos_teste, previsoes,
labels=orange(n_pessoas))
confusoes = [(i, j, matriz[i, j]) for i in range(n_pessoas) for j in
range(n_pessoas)
              if i != j and matriz[i, j] > 0]
confusoes.sort(key=lambda x: x[2], reverse=True)
```

Resultado obtido: dos 6 erros, nenhum par de pessoas se repetiu — os erros parecem isolados/marginais, não uma limitação estrutural do método para pares específicos de rostos parecidos (diferente do comportamento observado em teste sintético controlado, onde duas "pessoas" deliberadamente parecidas geraram confusão sistemática e repetida quando poucos componentes eram usados).

4.5 Comparação com SVD direta (`comparar_svd.py`)

```
U, valores_singulares, _ = np.linalg.svd(A_centralizada,
full_matrices=False)
autovalores_svd = (valores_singulares ** 2) / (n_amostras - 1)

sinais = np.sign(np.sum(autovetores_pca * autovetores_svd, axis=0))
sinais[sinais == 0] = 1
autovetores_svd_alinhados = autovetores_svd * sinais
```

Ver seção 3.9 para a discussão teórica e os resultados numéricos completos.

4.6 Demonstração concreta de uma classificação individual

Célula adicional criada para tornar o mecanismo de decisão explícito e visual (não apenas a métrica agregada de acurácia): pega uma foto de teste específica, calcula a distância dela até **todas** as fotos de treino no espaço de 50 componentes, e exibe lado a lado a foto de teste e as 5 fotos de treino mais próximas, com a distância de cada uma e indicação visual (verde/vermelho) de acerto.

```
def testar_uma_classificacao(indice_teste, A_treino, A_teste, media_treino,
                             autovetores,
                             rotulos_treino, rotulos_teste, altura,
                             largura, top_n=5):
    vetor_teste = projetar(A_teste[:, [indice_teste]], media_treino,
                             autovetores[:, 0])
    coef_treino_local = projetar(A_treino, media_treino, autovetores)
    distancias = np.linalg.norm(coef_treino_local - vetor_teste[:, None],
                                axis=0)
    ordem = np.argsort(distancias)
    pessoa_real = rotulos_teste[indice_teste]
    pessoa_prevista = rotulos_treino[ordem[0]]
    # ... impressão e visualização das top_n fotos de treino mais próximas
```

Usa `autovetores_treino[:, :n_componentes_clf]` — ou seja, **50 eigenfaces**, a mesma redução de dimensionalidade usada na classificação agregada.

5. Bugs encontrados durante o desenvolvimento (e por que são relevantes)

5.1 Centralização dupla no cálculo do erro de reconstrução

Sintoma: o gráfico de erro vs. número de componentes estava **subindo** em vez de descer — matematicamente impossível para uma implementação correta, dado o Teorema de Eckart-Young (seção 3.7).

Causa: `projetar()` já subtrai a média internamente. O código passava `A_centralizada` (que já tinha subtraído a média uma vez) para dentro dessa função, que subtraía de novo — a média sendo removida duas vezes, deslocando a reconstrução proporcionalmente e de forma crescente com o número de componentes usados.

Correção: passar sempre `A` (dados originais, não centralizados previamente) para `projetar()`, já que ela centraliza internamente.

Por que isso importa para a defesa: o bug foi identificado justamente porque o resultado contradizia uma propriedade teórica conhecida — bom exemplo de como validar implementações não é só "rodar sem erro", mas conferir se os resultados respeitam as garantias matemáticas esperadas.

5.2 Limiar rígido demais na comparação PCA vs. SVD

Sintoma: a comparação com dados reais (rostos) acusou "diferença maior que o esperado" com um limiar fixo de `1e-6`, mesmo a implementação estando correta.

Causa: o limiar havia sido calibrado com dados sintéticos bem-condicionados (diferença $\sim 10^{-13}$). Dados reais de rosto são mais correlacionados, elevando o número de condição de A e, consequentemente, a diferença numérica esperada entre as duas rotas (seção 3.9).

Correção: substituição do veredito binário por uma interpretação em 3 níveis, incluindo o cálculo explícito do número de condição de A e $A^T A$ para contextualizar a diferença observada como consistente com a teoria, e não como erro.

5.3 Confusão de nomenclatura (**predicoes** vs. **precisoes**)

Sintoma: `ValueError: Found input variables with inconsistent numbers of samples: [100, 10]` na célula da matriz de confusão.

Causa: duas variáveis com propósitos diferentes (o vetor de 100 previsões da classificação, e a lista de 10 acurácias do experimento "acurácia vs. rank") tinham nomes visualmente parecidos o suficiente para serem confundidos ao editar o notebook.

Correção: padronização de nomenclatura no notebook final — **previsoes** (vetor de 100, usado na matriz de confusão) e **lista_acuracias/precisoes** (lista de 10, usada só no gráfico de acurácia vs. rank), com nomes propositalmente distintos.

6. Limitações do sistema

1. **Não há rejeição de "pessoa desconhecida":** o classificador de vizinho mais próximo sempre atribui a foto de teste à pessoa mais parecida entre as 40 do treino, mesmo que a foto seja de alguém completamente diferente. Não existe um limiar de distância para indicar "não reconheço essa pessoa".
2. **Dataset controlado:** Olivetti já vem com rostos centralizados, mesmo tamanho, iluminação relativamente controlada. Fotos do mundo real exigiriam pré-processamento (detecção facial + alinhamento) não implementado neste projeto.
3. **k-NN com k=1 é sensível a outliers:** uma única foto de treino atípica pode distorcer a classificação de fotos de teste legítimas.
4. **Método é linear:** PCA só captura variação linear entre os dados; variações não-lineares (ex: rotação de pose em 3D) não são bem representadas por um espaço linear reduzido.

7. Resultados consolidados

Métrica	Valor
Total de componentes possíveis	até 399
Componentes para 90% da variância	36
Componentes para 95% da variância	38
Componentes para 99% da variância	177
Acurácia da classificação (50 componentes)	94%
Erros na matriz de confusão	6 de 100 (nenhum par repetido)

Métrica	Valor
Número de condição de A	$\approx 4,4 \times 10^5$
Número de condição de $A^T A$	$\approx 1,93 \times 10^{11}$
Diferença numérica PCA vs. SVD	$\approx 10^{-6}$ (consistente com a teoria)
Tempo PCA via $A^T A$ vs. SVD direta	PCA $\sim 3\text{-}6\times$ mais rápido

8. Perguntas de banca antecipadas e respostas diretas

Por que 50 componentes na classificação? Escolha empírica validada pelo gráfico de acurácia vs. rank: a partir de $\sim 20\text{-}30$ componentes a acurácia já satura perto do máximo; 50 dá margem confortável.

O que aconteceria com k maior que 399? Não é possível — o posto máximo de A centralizada é $\min(4096, 400) - 1 = 399$, pela restrição de centralização (soma das colunas centralizadas é sempre zero).

Por que distância euclidiana? É a métrica natural para k -NN em espaços vetoriais contínuos, e é a mesma norma (Frobenius/ L_2) já usada na teoria de mínimos quadrados e na aproximação de posto baixo do PCA.

Isso escala para datasets muito maiores? A rota via $A^T A$ escala bem enquanto n (amostras) for pequeno comparado a pixels; se n crescesse muito, SVD ou métodos incrementais/randomizados de PCA seriam preferíveis.

Como o treino influencia o teste, matematicamente? As eigenfaces e a média são parâmetros aprendidos exclusivamente do treino; o teste reaproveita esses mesmos parâmetros para projetar fotos novas, avaliando se a base generaliza — nunca há recomputação de PCA ou centralização a partir dos dados de teste (evita vazamento de dados).

$A^T A$ reduz a dimensionalidade para 50? Não — $A^T A$ é só o mecanismo computacional que permite calcular a base completa de até 399 eigenfaces de forma barata. A redução de dimensionalidade em si é a truncagem posterior e independente, escolhendo usar só as primeiras k eigenfaces dessa base completa.

Por que a covariância é dividida por $(n-1)$ e não por n ? Correção de Bessel — a média usada na centralização foi estimada dos próprios dados, consumindo um grau de liberdade. Note que essa escolha não afeta os autovetores (eigenfaces) em si, só a escala numérica dos autovalores — relevante para interpretação de variância explicada e para a fórmula de conversão com SVD.

9. Roteiro de apresentação (fala)

(condensado; ver discussão completa na conversa original para a versão estendida seção por seção)

1. **Abertura:** apresentar o problema (reconhecimento facial via Eigenfaces/PCA) e o roteiro da apresentação.
2. **Representação dos dados:** explicar a vetorização de imagens em A (4096×400).
3. **Centralização:** rosto médio e por que subtrair a média é necessário para PCA.

4. **PCA via autovalores de $A^T A$** : o núcleo matemático — apresentar a prova da relação entre autovetores de $A^T A$ e AA^T .
 5. **Eigenfaces**: mostrar a visualização e explicar que não são "rostos reais".
 6. **Variância explicada**: quantos componentes bastam para reter a informação.
 7. **Reconstrução**: aproximação de posto baixo, Teorema de Eckart-Young.
 8. **Classificação**: metodologia treino/teste, o que o treino "ensina" ao modelo.
 9. **Demonstração concreta**: uma classificação passo a passo (seção 8.1 do notebook).
 10. **Acurácia vs. componentes e matriz de confusão**.
 11. **Comparação com SVD**: validação cruzada e discussão de número de condição.
 12. **Conclusões e limitações**.
-

10. Anexo — Notas operacionais do desenvolvimento

- **Ambiente**: Arch Linux, venv Python (`python -m venv venv`), dependências via `pip install -b --break-system-packages -r requisitos.txt` (necessário pela proteção PEP 668 do Python gerenciado pelo Arch).
 - **Problema de acesso ao dataset no Google Colab**: `fetch_olivetti_faces()` pode falhar com `HTTPError 403: Forbidden` devido a bloqueios intermitentes do Figshare (problema documentado e recorrente em issues do próprio `scikit-learn`, afetando Colab, Azure e GitHub Actions). **Solução aplicada**: exportar o dataset já em cache local para um arquivo `.npz` (`np.savez_compressed()`), fazer upload manual desse arquivo no Colab, e carregar via `np.load()` em vez de `fetch_olivetti_faces()` — elimina completamente a dependência de rede externa durante a apresentação.
 - **Controle de versão**: repositório Git com commits separados por mudança lógica (features vs. correções de bugs), incluindo mensagens detalhadas documentando a causa raiz de cada bug corrigido.
-

11. Conclusão

O projeto demonstra, de ponta a ponta, como PCA e SVD permitem comprimir dados de altíssima dimensão (4096 pixels) para representações muito menores (~50 números) preservando informação suficiente para uma tarefa de classificação com alta acurácia (94%). Além da aplicação prática, o projeto explora a fundo os aspectos teóricos por trás de cada escolha de implementação: o truque computacional de trabalhar com $A^T A$ em vez de AA^T , a correção estatística de Bessel na normalização da covariância, a garantia teórica (Eckart-Young) de que a aproximação de posto baixo é ótima, e a análise de estabilidade numérica comparando duas rotas de cálculo matematicamente equivalentes mas com propriedades numéricas distintas — validada tanto teoricamente quanto empiricamente, com números de condição concretos.